Draw It or Lose It

CS 230 Project Software Design Template

Version 1.0

Table of Contents

CS 230 Project Software Design Template 1

Table of Contents 2
Document Revision History 2
Executive Summary 3
Requirements 3
Design Constraints 3
System Architecture View 3
Domain Model 3
Evaluation 4
Recommendations 5

Document Revision History

Version	Date	Author	Comments
1.0	03/21/26	Mohammed Suleiman	Initial creation of the software design document

Instructions
Fill in all bracketed information on page one (the cover page), in the Document Revision History table, and below each header. Under each header, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Executive Summary

The Gaming Room currently offers their popular game Draw It or Lose It exclusively as an Android application. To increase the market reach, the client wants to develop a web based version of the game that serves multiple platforms. However, the clients staff lacks the technical knowledge to establish this new environment. To solve this, our consulting team is streamlining the development by designing a Java based backend architecture. This solution will implement specific design patterns and object oriented programming principles to ensure critical software requirements are met, such as enforcing unique game and team names and ensuring only one instance of the game engine exists in memory at any given time.

Requirements

< Please note: While this section is not being assessed, it will support your outline of the design constraints below. *In your summary, identify each of the client's business and technical requirements in a clear and concise manner.>*

Design Constraints

Developing this game for a web based distributed environment introduces several design constraints. The first constraint is platform independence, as the game must transition from and Android only app to a multi platform web application. This requires the backend to be developed using a portable language that can communicate seamlessly with various front end web clients. The second constraint involves memory management, as the requirement constrains the architecture, necessitating the implementation of Singleton design pattern within the GameService class to prevent duplicate or conflicting game registries across the environment. Finally, the third constraint is data integrity and uniqueness. Game and team names must be strictly unique to allow users to verify name availability. This constrains how data is added to the system, requiring the implementation of the iterator pattern to efficiently scan existing records and prevent duplicate entries before saving new data.

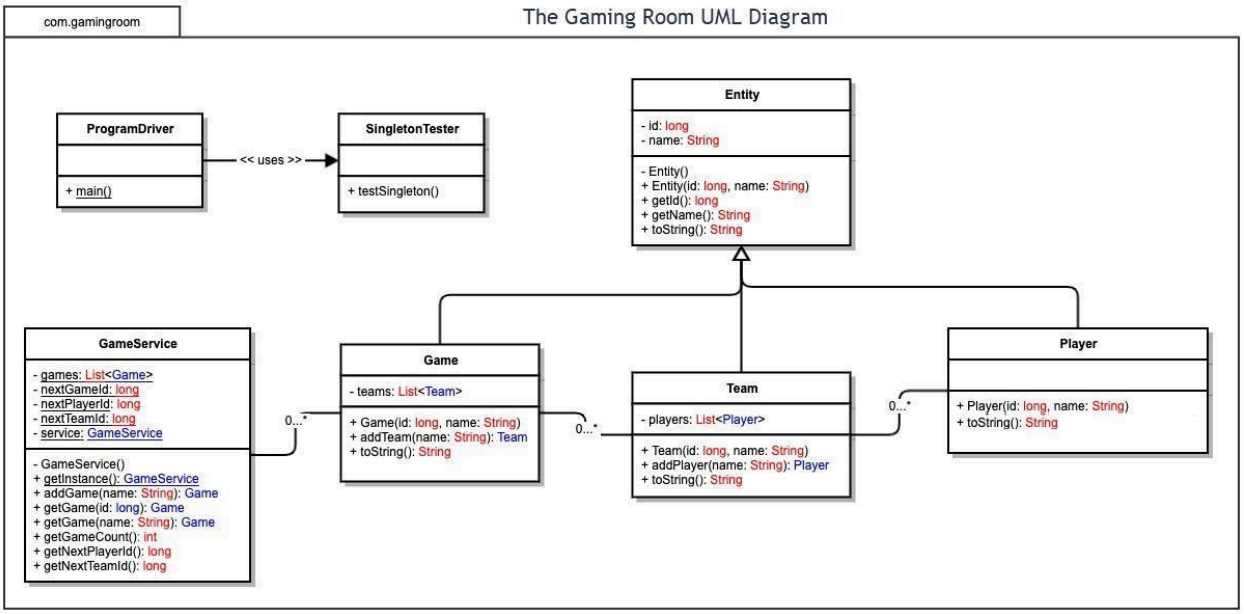
System Architecture View

Please note: There is nothing required here for these projects, but this section serves as a reminder that describing the system and subsystem architecture present in the application, including physical components or tiers, may be required for other projects. A logical topology of the communication and storage aspects is also necessary to understand the overall architecture and should be provided.

Domain Model

The provided UML diagram illustrates the structural relationships and object oriented programming principles used to efficiently fulfill the software requirements. In terms of class relationships, the ProgramDriver serves as the entry point and uses the SingletonTester to verify the environment, while the GameService class acts as the central engine holding lists of active games, teams, and players. The Game class has a one to many relationship with the Team class, meaning a single game can contain multiple teams, and similarly, the Team class has a one to many relationship with the Player class. This model heavily demonstrates the principle of inheritance through the introduction of the Entity base class. Because every entity in the application requires a unique identifier and a name, the Entity class encapsulates these attributes so the Game, Team, and Player classes can all inherit from this single parent class. This fulfills the software requirements by reducing code duplication and establishing a

standardized data structure. The diagram also demonstrates encapsulation by keeping class attributes private, indicated by the minus sign, and exposing them through public methods which protects the internal state of the objects from unintentional external interference.



Evaluation

Using your experience to evaluate the characteristics, advantages, and weaknesses of each operating platform (Linux, Mac, and Windows) as well as mobile devices, consider the requirements outlined below and articulate your findings for each. As you complete the table, keep in mind your client's requirements and look at the situation holistically, as it all has to work together.

In each cell, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Development Requirements	**Mac**	**Linux**	**Windows**	**Mobile Devices**

Server Side	MacOs can run web server software, but it is traditionally not utilized nor designed for enterprise web hosting. Apple discontinued its dedicated macOS Server software, meaning hosting would rely on configuring third party tools on Mac metal. The primary weakness is that it requires proprietary and expensive Apple hardware to run, making scaling it to thousands of players inefficient in a data center. While there are no direct licensing costs for the OS itself, the hardware premium makes this unviable and extremely costly.	Linux is the standard OS for enterprise web hosting and is highly recommended for this project. It offers a lightweight and highly secure server based deployment method that can scale to accommodate thousands of players concurrently. The primary advantage is the cost efficiency, as most distros are open source and free, meaning we could incur zero licensing fees for the OS. The main weakness is the steeper learning curve, as it requires system administrators with command line knowledge rather than relying on GUIs.	Windows Server is a capable, enterprise ready OS that can offer strong server based deployment method through Internet Information Services. Its primary advantage is its user friendly GUI and integration with other Microsoft enterprise products. A weakness is its heavy resource footprint when compared to Linux. Windows Server also requires a license which would add ongoing license fees to the budget.	Mobile platforms are designed strictly as client-side OSs and do not offer a server based deployment method. They lack the computing power, continuous network stability and architectural design required to host an application or database. There is no enterprise server software designed to run on mobile devices. They are unviable for hosting the server side infrastructure of a game for thousands of players and licensing costs are not applicable.

Client Side	The development process must focus on browser compatibility, ensuring HTML interface runs on Safari and Google Chrome for macOS. This requires standard front end web development using HTML/CSS/JavaScript rather than Apple programming languages like Swift. Because the client is web based, the development time and cost are significantly reduced compared to building a desktop application, as the same front end codebase can be shared across other desktop OS.	Supporting Linux clients requires ensuring the web application functions on browsers frequently used by the Linux community, like Mozilla Firefox and Chromium/Chrome. This requires expertise in front end web development and REST API integration. The cost and time required are optimized as long as the development team adheres strictly to W3C standards.	The web application must be tested for compatibility with desktop browsers like Microsoft Edge, Chrome, and Firefox. The required expertise is centered on modern web development frameworks that can communicate effectively with the backend REST API. By utilizing a web based client rather than a native one, the development time and costs are minimized. The client avoids the need to hire specialized .NET or C# developers, needing only a single shared codebase to serve the user base.	Supporting mobile platforms introduces the most considerations. The development process requires strict adherence to responsive web design principles to ensure the HTML interface scales seamlessly to smaller touchscreens and varying device orientations. The required expertise expands to include mobile UX/UI design, touch events, and mobile browser optimization. While developing a single web app is still less expensive and time consuming than building separate native apps, it demands a higher allocation of budget and testing time compared to desktop platforms.

Development Tools	To build this client for macOS, the primary programming languages are HTML/CSS/JS. Developers will use standard IDEs like VSCode alongside Safari browser for local testing. Because the app is web based, there is no need for a development team who can code in Swift/Objective-C. There are no licensing costs for the development tools, but proprietary Mac hardware will be needed for testing Safari compatibility.	Development for Linux will rely on the same core web languages on the client side, and Java for the backend server side. The development team can use Eclipse IDE or VSCode. The development team coding the Windows or Mac web clients can build the Linux version and because the OS, programming languages and IDEs are open source, there are no licensing fees.	For Windows environments, developers will use HTML/CSS/JS, and utilize IDEs like VSCode. The project does not require a team of specialized .NET or C# developers. Enterprise versions of Visual Studio do have subscription fees but the team can use VSCode for free to complete the web app.	iOS and Android require advanced usage of HTML/CSS/JS focusing on mobile UI frameworks, touch events and responsive media queries. The tools include device emulators like Android Studio and Apple Xcode. Because the app is web based and not native, there is less expense incurred as there isn't a need for separate Swift and Kotlin development teams. Android Studio and Xcode are free, but testing iOS web apps requires Mac hardware.

Recommendations

Analyze the characteristics of and techniques specific to various systems architectures and make a recommendation to The Gaming Room. Specifically, address the following:

1. **Operating Platform**: To expand Draw It or Lose It to multiple environments, I recommend a Linux based operating system for the back end environment. Because the client application will be delivered as a responsive HTML interface in web browsers across multiple different operating systems and devices, the server must be an extremely scalable and cost effective one. Linux is the industry standard for hosting distributed web applications and RESTful APIs. It is open source, incurring no licensing fees as we scale the server infrastructure to support thousands of concurrent players.

2. **Operating Systems Architectures**: The operating system operates in the kernel space, managing the memory, CPU and processes directly in a monolithic kernel architecture, providing high performance and hardware interaction speeds. Linux separates this kernel space from the user space where the game and its database will run. If the application crashes or encounters a fatal error, that isolation ensures that the core operating system remains online, preventing a server collapse.

3. **Storage Management**: For Linux on the logical and local software side of storage management, the native ext4 filesystem is an appropriate system due to its reliability. For the hardware, the server should utilize Solid State Drives to ensure maximum read/write speeds. Because the game requires 1.6 GB of HD images, a hybrid architecture should be used. The local ext4 filesystem should have the application's codebase and database living there, while the HD images should be stored to a cloud-based storage system to prevent the local SSDs from reaching capacity and to utilize a Content Delivery Network for fast image delivery.

4. **Memory Management**: Linux uses an efficient virtual memory system with demand paging, mapping virtual addresses used by the application to physical RAM. If the physical memory becomes full, like when too many game instances are running at the same time, Linux will swap the inactive data from RAM to a designated spot on the hard drive. For this game, which must render 8 MB images during 1 minute rounds, this means the OS will prioritize keeping the active image assets and the Java REST API in RAM, while paging out idle background processes, preventing out of memory crashes.

5. **Distributed Systems and Networks**: The game will operate as a client-server distributed system. The multiple client devices will communicate with the central Linux server via a REST API over standard internet protocols. This network dependency means the game requires constant internet connectivity. If a network outage occurs, the client devices will lose the ability to fetch new images or submit guesses. To mitigate the impact of server side outages, the distributed architecture should utilize a load balancer to distribute incoming player traffic across multiple Linux servers, ensuring high availability even if one server node goes offline.

6. **Security**: On the Linux server, native security capabilities like iptables for firewall configuration and SELinux will be used to enforce Mandatory Access Control, ensuring that even if a vulnerability is found, an attacker cannot easily access the underlying system files. For crossplatform communication, all data in transit between the client web browsers and the server

will be encrypted using TLS/SSL protocols. To handle authorization securely without managing complex session states, the application will use stateless JSON Web Tokens, self contained tokens for storing user data within the token itself, and Role Based Access Control, a model to restrict system access to authorized users based on their defined role, guaranteeing that users can only access endpoints and data appropriate for their specific permission level.